- Speeds up the development process and provides a fast feedback loop because contract tests don’t need to communicate to one or more systems
- Enables teams to deploy changes independently and safely
- Makes problems easier to debug because the scope of a contract test is much smaller
- Enables developers to run contract tests and update them locally when needed, creating a shared responsibility when it comes to writing tests
- Promotes better code quality because it removes the need to introduce requirements that won’t be used
- Ensures that any breaking changes are communicated to teams because the contract will always be up-to-date with these changes, especially with contract testing tools that generate a contract from the development codebase

1.3 When is contract testing appropriate to use?

Contract testing is appropriate to use whenever you need to test an integration point between two systems for which you want to catch problems early. This could be between a web (or mobile) application and an API or between two data services, communicating via different protocols, with HTTP being the most common one.

Although you can use contract testing when working with just two different systems that communicate with each other, it shines the most within a microservices architecture, especially if it involves a large number of microservices. Relying on API integration tests or E2E tests alone will be difficult to manage and scale up. Contract testing provides a great alternative.

> **Tip** Contract testing can be used between two connected systems. However, the same principles will apply when you write a contract between a web application communicating to multiple microservices.

In some scenarios, contract testing might not be appropriate to use, such as the following: